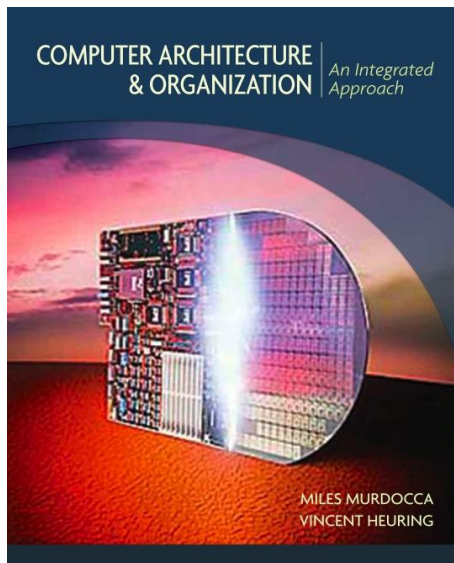


Organización de las Computadoras

Carolina Chaves Garcia



GPIO en ESP32-C3

Introducción

Introducción

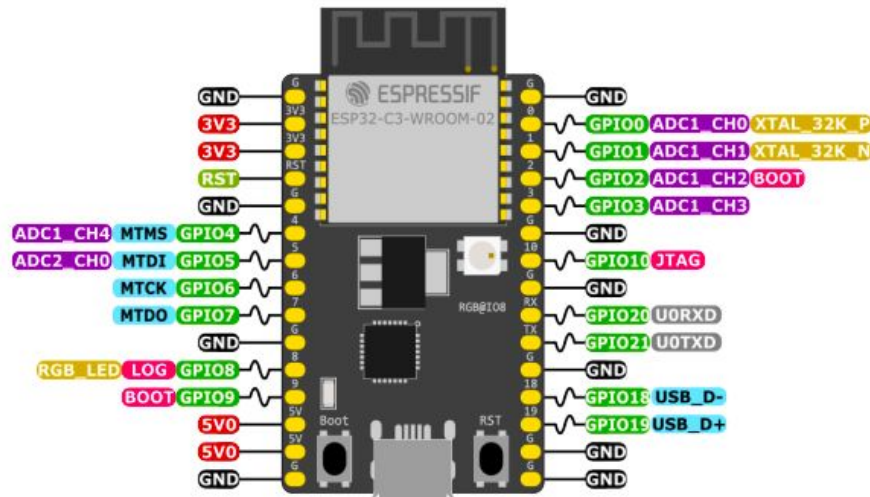
GPIO

- **General-Purpose Input/Output** = GPIO = Entrada/Salida de Propósito General. También le solemos decir "Pin de Propósito General".
- Trabajan generalmente con dos estados:
 - Alto (High / 1)
 - Bajo (Low / 0)
- Cumple dos funciones básicas:
- Como **Entrada (Input)**: Para **leer** o **escuchar** una señal del exterior. Por ejemplo:
 - Leer el estado de un pulsador o interruptor.
 - Detectar un sensor.
 - Medir distancia con un sensor de ultrasonidos.
 - etc.
- Como **Salida (Output)**: Para **enviar** o **dar** una señal al exterior. Por ejemplo:
 - Encender y apagar un LED.
 - Controlar un altavoz para generar sonidos.
 - Activar un relé para controlar dispositivos de alto voltaje (como una lámpara de casa).
 - Controlar servomotores (servos) o motores paso a paso.

Introducción

GPIO ESP32-C3

- El ESP32-C3 tiene 22 pines GPIO multifunción que pueden configurarse como:
 - Entradas digitales
 - Salidas digitales
 - Periféricos especiales (UART, SPI, I2C, etc.)



ESP32-C3 Specs

32-bit RISC-V single-core @160MHz
 Wi-Fi IEEE 802.11 b/g/n 2.4GHz
 Bluetooth LE 5
 400 KB SRAM (16 KB for cache)
 384 KB ROM
 22 GPIOs, 3x SPI, 2x UART, I2C,
 I2S, RMT, LED PWM, USB Serial/JTAG,
 GDMA, TWAI®, 12-bit ADC

PWM Capable Pin

- Miscellaneous/Secondary Functions
- General Purpose Input and Output
- Other Related Functions
- JTAG for Debugging and/or USB
- Analog-to-Digital Converter
- Serial for Debug/Programming
- Ground Plane
- Power Rails (3V3 and 5V)
- Strapping Pin Functions

Introducción

GPIO ESP32-C3

- En la tabla 2-4 del datasheet encontramos las funciones de los pines.

Pin No.	IO MUX / GPIO Name ²	IO MUX Function ^{1, 2, 3}					
		F0	Type ³	F1	Type	F2	Type
4	GPIO0	GPIO0	I/O/T	GPIO0	I/O/T		
5	GPIO1	GPIO1	I/O/T	GPIO1	I/O/T		
6	GPIO2	GPIO2	I/O/T	GPIO2	I/O/T	FSPIQ	I1/O/T
8	GPIO3	GPIO3	I/O/T	GPIO3	I/O/T		
9	GPIO4	MTMS	I1	GPIO4	I/O/T	FSPIHD	I1/O/T
10	GPIO5	MTDI	I1	GPIO5	I/O/T	FSPIWP	I1/O/T
12	GPIO6	MTCK	I1	GPIO6	I/O/T	FSPICLK	I1/O/T
13	GPIO7	MTDO	O/T	GPIO7	I/O/T	FSPID	I1/O/T
14	GPIO8	GPIO8	I/O/T	GPIO8	I/O/T		
15	GPIO9	GPIO9	I/O/T	GPIO9	I/O/T		
16	GPIO10	GPIO10	I/O/T	GPIO10	I/O/T	FSPICSO	I1/O/T
18	GPIO11	GPIO11	I/O/T	GPIO11	I/O/T		
19	GPIO12	SPID	I1/O/T	GPIO12	I/O/T		
20	GPIO13	SPIWP	I1/O/T	GPIO13	I/O/T		
21	GPIO14	SPICSO	O/T	GPIO14	I/O/T		
22	GPIO15	SPICLK	O/T	GPIO15	I/O/T		
23	GPIO16	SPID	I1/O/T	GPIO16	I/O/T		
24	GPIO17	SPIQ	I/O/T	GPIO17	I/O/T		
25	GPIO18	GPIO18	I/O/T	GPIO18	I/O/T		
26	GPIO19	GPIO19	I/O/T	GPIO19	I/O/T		
27	GPIO20	UORXD	I1	GPIO20	I/O/T		
28	GPIO21	UOTXD	O	GPIO21	I/O/T		

- Pines IO: asignados para la comunicación con la memoria flash integrada y **NO** recomendados para otros usos.

- Pines IO: tienen una de funciones importantes.

Memoria Mapeada y Registros Especiales

Memoria Mapeada y Registros Especiales

- El microcontrolador tiene una dirección de memoria especial para cada función (GPIO, UART, Timers, etc)
- Escribir en esa dirección no guarda un dato, sino que configura el hardware.
- Esas direcciones de memoria especiales son los **registros de periféricos**. En el Manual de Referencia Técnica del ESP32-C3 encontramos estos registros. Siempre debemos saber qué registro usar y qué valor debemos escribir en él.

Name	Description	Address	Access
Configuration Registers			
GPIO_BT_SELECT_REG	GPIO bit select register	0x0000	R/W
GPIO_OUT_REG	GPIO output register	0x0004	R/W/SS
GPIO_OUT_W1TS_REG	GPIO output set register	0x0008	WT
GPIO_OUT_W1TC_REG	GPIO output clear register	0x000C	WT
GPIO_ENABLE_REG	GPIO output enable register	0x0020	R/W/SS
GPIO_ENABLE_W1TS_REG	GPIO output enable set register	0x0024	WT
GPIO_ENABLE_W1TC_REG	GPIO output enable clear register	0x0028	WT
GPIO_STRAP_REG	pin strapping register	0x0038	RO
GPIO_IN_REG	GPIO input register	0x003C	RO
GPIO_STATUS_REG	GPIO interrupt status register	0x0044	R/W/SS
GPIO_STATUS_W1TS_REG	GPIO interrupt status set register	0x0048	WT
GPIO_STATUS_W1TC_REG	GPIO interrupt status clear register	0x004C	WT
GPIO_PRO_CPU_INT_REG	GPIO PRO_CPU interrupt status register	0x005C	RO
GPIO_STATUS_NEXT_REG	GPIO interrupt source register	0x014C	RO
Pin Configuration Registers			
GPIO_PIN0_REG	GPIO pin0 configuration register	0x0074	R/W
GPIO_PIN1_REG	GPIO pin1 configuration register	0x0078	R/W

Registro para escribir el estado de salida (1/0) de todos los pines.

Registro para leer el estado de entrada de todos los pines.

Registro para habilitar un pin como salida.

GPIO_FUNCx_OUT_SEL_CFG_REG: Registros para seleccionar la función del pin.

GPIO de salida

GPIO como salida

Ejemplo 1: Encender y apagar un LED

- Vamos a **enviar una señal al exterior** para poder **encender el LED**, por lo tanto debemos configurar uno de los pines como **salida**.
- Revisar en el [Datasheet](#) o en el [Manual Técnico](#) para encontrar la dirección base de los GPIO.



- Vamos a usar un GPIO para conectar el LED. ¿Cuál es recomendable usar?
- Escogemos el GPIO1 para conectar el LED.
- Definimos los registros GPIO1
 - GPIO_BASE_ADDR 0x60004000
 - GPIO_ENABLE_REG 0X0020
 - GPIO_OUT_REG 0x0004
 - GPIO_OUT_W1TS_REG 0x0008
 - GPIO_OUT_W1TC_REG 0x000C
- Agregamos una máscara para poder poner el pin 1 como salida
 - GPIO1_MASK, (1 << 1)

GPIO como salida

Ejemplo 1: Encender y apagar un LED

```
.section .text
.global configurar_gpio_asm
.global encender_led_asm
.global apagar_led_asm

.equ GPIO_BASE_ADDR, 0x60004000 # Dirección base periférico GPIO
.equ GPIO_ENABLE_REG, 0x0020    # Offset registro ENABLE
.equ GPIO_OUT_REG, 0x0004      # Offset registro OUTPUT
.equ GPIO_OUT_W1TS_REG, 0x0008  # Offset registro SET (poner a 1)
.equ GPIO_OUT_W1TC_REG, 0x000C  # Offset registro CLEAR (poner a 0)

.equ GPIO1_MASK, (1 << 1) # Máscara para GPIO1
```

- Configuraremos el GPIO, para ellos debemos habilitar el GPIO que deseamos.
- Debemos calcular la dirección base del GPIO.
- Habilitamos el registro y guardamos ese valor en memoria.

GPIO como salida

Ejemplo 1: Encender y apagar un LED

configurar_gpio_asm:

Calcular dirección del registro GPIO_ENABLE

li t0, GPIO_BASE_ADDR # t0 = 0x60004000

addi t0, t0, GPIO_ENABLE_REG # t0 = 0x60004000 + 0x0020 = 0x60004020

Preparar valor para habilitar GPIO1

li t1, GPIO1_MASK # t1 = 0b00000010 (bit 1 en 1)

Escribir en registro GPIO_ENABLE

sw t1, 0(t0) # Habilitar GPIO1 como salida

ret # Retornar a C

- Para encender el LED, tenemos dos opciones:
 - Opción 1: Usaremos el registro GPIO_OUT_W1TS_REG
 - Solo pone a 1 los bits que están en 1 en la máscara
 - Los demás bits permanecen inalterados
 - Operación atómica - **más segura**
 - Opción 2: Usamos el registro OUTPUT directamente
 - Sobreescribe **TODOS** los bits del registro
 - Si quieres cambiar solo GPIO1, debes leer-modificar-escribir
 - Peligroso si otros GPIOs están en uso

GPIO como salida

Ejemplo 1: Encender y apagar un LED

- Vamos a crear la función para configurar el LED (encender_led_asm)
- Opción 1: Usaremos el registro GPIO_OUT_W1TS_REG

encender_led_asm:

Calcular dirección del registro SET para encender GPIO01 (High)

li t0, GPIO_BASE_ADDR # t0 = 0x60004000

addi t0, t0, GPIO_OUT_W1TS_REG # t0 = 0x60004008

Preparar máscara para GPIO1

li t1, GPIO1_MASK # Máscara para GPIO3

Escribir en registro SET (pone a 1 solo el bit especificado)

sw t1, 0(t0) # Encender LED en GPIO3

ret # Retornar a C

- Opción 2: Usamos el registro OUTPUT directamente

li t0, GPIO_BASE_ADDR

addi t0, t0, GPIO_OUT_REG # t0 = 0x60004004

li t1, GPIO1_MASK

sw t1, 0(t0) # También enciende el LED

GPIO como salida

Ejemplo 1: Encender y apagar un LED

- Por último realizamos el llamado de la función desde C

```
extern void configurar_gpio_asm(void);
extern void encender_led_asm(void);
extern void apagar_led_asm(void);

void app_main(void)
{
    printf("Probando LED RGB en GPIO1\n");
    configurar_gpio_asm();

    while(1) {
        printf("GPIO1 = HIGH\n");
        encender_led_asm(); // GPIO1 = 1
        vTaskDelay(pdMS_TO_TICKS(2000));

        printf("GPIO1 = LOW\n");
        apagar_led_asm(); // GPIO1 = 0
        vTaskDelay(pdMS_TO_TICKS(2000));
    }
}
```

GPIO de entrada

GPIO como entrada

Ejemplo 2: Encender y apagar un LED con pulsador

- Vamos a **recibir una señal al exterior**, por lo tanto debemos configurar uno de los pines como **entrada**.
- Revisar en el [Datasheet](#) o en el [Manual Técnico](#) para encontrar la dirección del registro encargado de las entradas: GPIO_IN_REG

GPIO_IN_REG	GPIO input register	0x003C
-------------	---------------------	--------

- Agregamos esta dirección a nuestro código, junto con la máscara correspondiente a su pin.

.section .text

```
.equ GPIO_BASE_ADDR, 0x60004000 # Dirección base periférico GPIO
.equ GPIO_ENABLE_REG, 0x0020    # Offset registro ENABLE
.equ GPIO_OUT_REG, 0x0004       # Offset registro OUTPUT
.equ GPIO_OUT_W1TS_REG, 0x0008   # Offset registro SET (poner a 1)
.equ GPIO_OUT_W1TC_REG, 0x000C   # Offset registro CLEAR (poner a 0)
.equ GPIO_IN_REG, 0x003C        # Offset del registro de entrada
```

```
.equ GPIO1_MASK, (1 << 1) # Máscara para GPIO1 (salida LED)
.equ GPIO3_MASK, (1 << 3) # Máscara para GPIO3 (entrada del pulsador)
```

GPIO como entrada

Ejemplo 2: Encender y apagar un LED con pulsador

- Esta vez vamos a crear una función donde hagamos todo, la llamaremos `leer_pulsador_y_controlar_led_asm`
- Primero configuraremos el GPIO3 como entrada

`leer_pulsador_y_controlar_led_asm:`

`li t2, GPIO_BASE_ADDR`

`addi t2, t2, GPIO_IN_REG      # Dirección de GPIO_IN (lectura de entradas)`

`li t3, GPIO3_MASK              # Máscara para GPIO3 (botón)`

- Luego configuramos el GPIO1 como salida

`li t4, GPIO_BASE_ADDR`

`addi t4, t4, GPIO_OUT_W1TS_REG # Dirección para SET GPIO1 (encender LED)`

`li t5, GPIO_BASE_ADDR`

`addi t5, t5, GPIO_OUT_W1TC_REG # Dirección para CLEAR GPIO1 (apagar LED)`

`li t6, GPIO1_MASK              # Máscara para GPIO1 (LED)`

GPIO como entrada

Ejemplo 2: Encender y apagar un LED con pulsador

- Debemos leer constantemente el pulsador para conocer su estado y así poder tomar decisiones.

loop_pulsador:

```
lw t0, 0(t2)      # Leer GPIO_IN  
and t1, t0, t3     # t1 = t0 & GPIO3_MASK
```

```
beq t1, x0, apagar_led    # Si t1 == 0, botón NO presionado → apagar LED
```

```
sw t6, 0(t4)          # Botón presionado → encender LED  
j loop_pulsador
```

- Luego creamos la función para apagar el LED

apagar_led:

```
sw t6, 0(t5)          # Apagar LED  
j loop_pulsador
```

GPIO como entrada

Ejemplo 2: Encender y apagar un LED con pulsador

- Por último realizamos el llamado de la función desde C

```
extern void leer_pulsador_y_controlar_led_asm(void);

void app_main(void)
{
    printf("Iniciando...\n");
    configurar_gpio_asm(); // Configura GPIO1 como salida

    printf("Esperando pulsador en GPIO3\n");
    leer_pulsador_y_controlar_led_asm(); // Bucle infinito en ASM
}
```

GPIO como entrada

Ejemplo 2: Encender y apagar un LED con pulsador

- Esto que acabamos de hacer se llama **polling**, es una técnica de programación en la que nuestro **microcontrolador revisa constantemente el estado de una entrada** (pulsador) para saber si ocurrió un evento.
- El **procesador está ocupado revisando constantemente** si el pulsador fue presionado.
- Ventajas de polling
 - Simple de implementar
 - No necesitas configurar interrupciones ni hardware adicional
- Desventajas de polling
 - Ineficiente: el CPU está ocupado preguntando todo el tiempo y no hace nada más útil.
 - No es adecuado si tienes muchos eventos diferentes que monitorear.
 - Puede provocar latencia si no se revisa con la frecuencia suficiente.
- **Forma correcta y segura: interrupciones**
 - El microcontrolador **no pregunta todo el tiempo**, sino que el hardware **interrumpe automáticamente al CPU** cuando ocurre un evento.

Gracias